

Lecture 6: Finding k -th minimum*Instructor: Sourav Chakraborty**Scribe: Ritam Ghosh, Barun Sau*

1 Motivation

In algorithm design, we often solve new problems by **reducing** them to problems we already know how to solve. The idea is to transform the current problem into a smaller version or into a known problem for which we have efficient algorithms.

2 Median Finding

Given an unsorted array $A = [a_1, a_2, \dots, a_n]$ where $a_i \in \mathbb{N}$, let $\hat{A} = [b_1, b_2, \dots, b_n]$ be the sorted version of A . The **median** is defined as:

$$\text{median}(A) = b_{\lceil n/2 \rceil}$$

2.1 Ways to Find the Median

We can brainstorm some ideas on how to find the median of the array such as:

- Repeatedly sample elements and track likely candidates.
- Recursively eliminate half of elements based on comparisons.
- Randomized pivot selection with recursion.

One naive approach can be just sort the array and then find the median (which is essentially the middle element in case of odd n or the average of $\lceil n/2 \rceil - 1$ th and $\lceil n/2 \rceil + 1$ th element).

Now to sort the array we need $\Theta(n \log n)$ time. But we just want to find the position of a median, so sorting the whole array is an overkill.

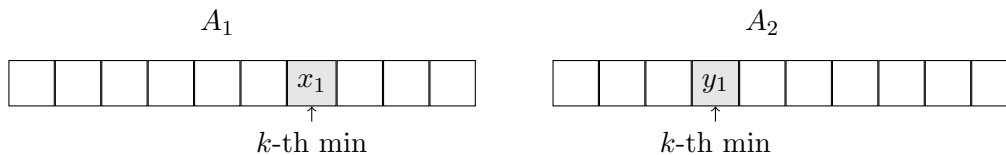
Why do we need $\Omega(n)$ comparisons?

- From a graph perspective, each comparison between elements can be represented as an edge in a graph.
- To determine the median, we need enough comparisons so that the comparison graph is connected.
- A connected graph on n vertices requires at least $n - 1$ edges.
- Therefore, $\Omega(n)$ comparisons are necessary in the comparison model.

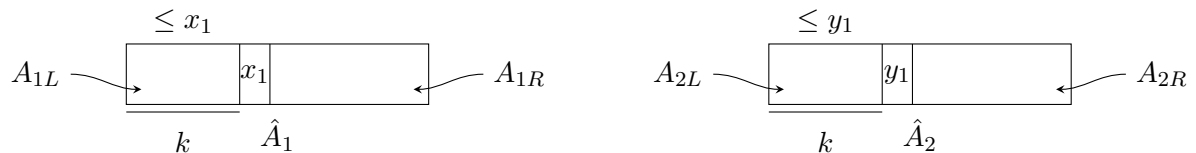
3 Finding the k -th Smallest Element:

A more general problem: Given array A and an integer k ($1 \leq k \leq n$), output b_k , the k -th smallest element of A .

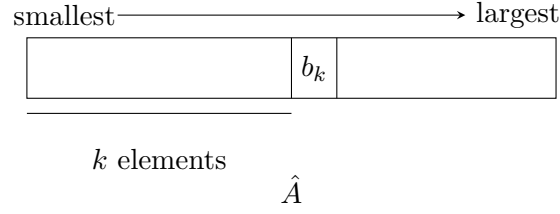
A common strategy is to break the array into two parts.



Now we find the k -th min in A_1 and k -th min in A_2 i.e we find out x_1 and y_1 . To visualize we can imagine the sorted version of A_1 as $\hat{A}_1$ and sorted version of A_2 as $\hat{A}_2$:



In both parts, we have the k -th minimums x_1 and y_1 . Our area of interest is the left part of x_1 and the left part of y_1 because in A we wish to find the k -th min which can be anywhere in A_{1L} or A_{2L} . If we see in $\hat{A}$:



$$k\text{-th min in } A = k\text{-th min in } A_{1L} \cup A_{2L}$$

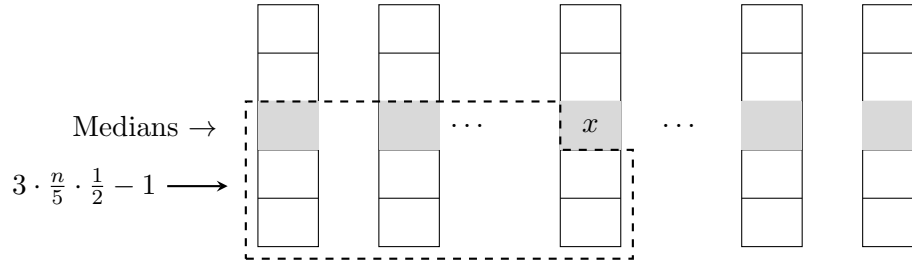
Thus the recurrence relation becomes:

$$T(n) = 2T(n/2) + T(2k) + O(n)$$

where:

- $T(2k)$ is the time required for finding the k -th min in $A_{1L} \cup A_{2L}$.
- $O(n)$ is the time required to find the elements lesser than the k -th min in each subarray.

Can we do better if we break the array into more than two parts? Let us break the array into multiple chunks where each chunk will have 5 elements of the array. So the array will be divided into $n/5$ parts.



Algorithm Description

1. Partition the array A into $\lceil n/5 \rceil$ groups $B_1, B_2, \dots, B_{\lceil n/5 \rceil}$, each of size at most 5.
2. Sort each group B_i individually.
3. Let y_i be the median of group B_i . Collect all medians into a new array

$$C = \{y_1, y_2, \dots, y_{\lceil n/5 \rceil}\}.$$

4. Recursively find the median of C :

$$x = \text{Find}(C, |C|/2).$$

This element x is called the *median of medians*.

5. Partition A using pivot x into:

$$A_L = \{a \in A \mid a < x\}, \quad A_R = \{a \in A \mid a > x\}.$$

6. Let $\text{pos}(x)$ be the rank of x in A .

- If $\text{pos}(x) = k$, return x .
- If $\text{pos}(x) > k$, recursively call $\text{Find}(A_L, k)$.
- If $\text{pos}(x) < k$, recursively call $\text{Find}(A_R, k - \text{pos}(x))$.

Why the Pivot is Good

Each group contributes at least 3 elements that are either $\leq y_i$ or $\geq y_i$.

Since x is the median of the medians:

- At least half of the medians are $\leq x$
- Each such median contributes at least 3 elements $\leq x$

Hence, at least

$$\frac{1}{2} \cdot \frac{n}{5} \cdot 3 = \frac{3n}{10}$$

elements are $\leq x$, and similarly at least $\frac{3n}{10}$ elements are $\geq x$.

Therefore, after partitioning, the larger recursive subproblem has size at most $\frac{7n}{10}$.

Running Time Analysis

Let $T(n)$ be the worst-case running time.

- Sorting all groups: $O(n)$
- Finding median of medians: $T(n/5)$
- Recursive selection after partitioning: $T(7n/10)$

Thus the recurrence is:

$$T(n) = T\left(\frac{n}{5}\right) + T\left(\frac{7n}{10}\right) + O(n)$$

Since:

$$\frac{n}{5} + \frac{7n}{10} < n,$$

by the Master Theorem or substitution method, we obtain:

$$T(n) = O(n).$$

Algorithm 1

Require: Array A of size n , integer k

Ensure: The k -th smallest element of A

```
1: if  $n = 1$  then
2:   return  $A[1]$ 
3: end if
4: Partition  $A$  into  $\lceil n/5 \rceil$  groups  $B_1, B_2, \dots, B_{\lceil n/5 \rceil}$  of size at most 5
5: for each group  $B_i$  do
6:   Sort  $B_i$ 
7:   Let  $y_i$  be the median of  $B_i$ 
8: end for
9: Construct array  $C = \{y_1, y_2, \dots, y_{\lceil n/5 \rceil}\}$ 
10:  $x \leftarrow \text{Find}(C, \lceil |C|/2 \rceil)$  ▷ Median of medians
11: Partition  $A$  into:


$$A_L = \{a \in A \mid a < x\}, \quad A_R = \{a \in A \mid a > x\}$$


12: Let  $p = |A_L| + 1$  ▷ Rank of  $x$  in  $A$ 
13: if  $p = k$  then
14:   return  $x$ 
15: else if  $k < p$  then
16:   return  $\text{Find}(A_L, k)$ 
17: else
18:   return  $\text{Find}(A_R, k - p)$ 
19: end if
```

Correctness Proof

We prove that the algorithm $\text{FIND}(A, k)$ correctly returns the k -th smallest element of array A .

Lemma 1 (Pivot Property). Let x be the median of medians chosen by the algorithm. Then at least $\frac{3n}{10}$ elements of A are less than or equal to x , and at least $\frac{3n}{10}$ elements of A are greater than or equal to x .

Proof. The array A is partitioned into $\lceil n/5 \rceil$ groups of size at most 5. Each group contributes one median to the array C .

Since x is the median of C , at least half of the medians are less than or equal to x . In each such group, at least 3 elements are less than or equal to its median. Therefore, the total number of elements less than or equal to x is at least

$$\frac{1}{2} \cdot \frac{n}{5} \cdot 3 = \frac{3n}{10}.$$

A symmetric argument shows that at least $\frac{3n}{10}$ elements are greater than or equal to x . $\square$

Lemma 2 (Problem Size Reduction). After partitioning around x , the size of the larger recursive subproblem is at most $\frac{7n}{10}$.

Proof. By Lemma 1, at least $\frac{3n}{10}$ elements are discarded from one side of the partition. Hence, the remaining elements passed to the recursive call are at most

$$n - \frac{3n}{10} = \frac{7n}{10}.$$

$\square$

Theorem (Correctness). The algorithm $\text{FIND}(A, k)$ returns the k -th smallest element of A .

Proof. The algorithm partitions A into A_L , $\{x\}$, and A_R , where all elements in A_L are smaller than x and all elements in A_R are larger than x .

Let $p = |A_L| + 1$ be the rank of x in A .

- If $p = k$, then x is the k -th smallest element and is returned.
- If $k < p$, the desired element lies in A_L , and the recursive call $\text{FIND}(A_L, k)$ is correct by induction.
- If $k > p$, the desired element lies in A_R , and the recursive call $\text{FIND}(A_R, k - p)$ is correct by induction.

Since the base case $n = 1$ is trivially correct, by induction the algorithm correctly returns the k -th smallest element for all n . □